

Document number	P0792R0
Date	2017-10-10
Reply-to	Vittorio Romeo < vittorio.romeo@outlook.com (mailto:vittorio.romeo@outlook.com)>
Audience	Library Evolution Working Group
Project	ISO JTC1/SC22/WG21: Programming Language C++

function_ref: a non-owning reference to a Callable

Abstract

This paper proposes the addition of `function_ref<R(Args...)>` to the Standard Library, a *"vocabulary type"* for non-owning references to `Callable` objects.

Table of contents

- [Overview](#)
- [Motivating example](#)
- [Impact on the Standard](#)
- [Alternatives](#)
- [Synopsis](#)
- [Specification](#)
- [Existing practice](#)
- [Possible issues](#)
- [Open questions](#)
- [Bikeshedding](#)
- [Acknowledgments](#)
- [References](#)

Overview

Since the advent of C++11 writing more functional code has become easier: functional programming patterns and idioms have become powerful additions to the C++ developer's toolbox. "**Higher-order functions**" are one of the key ideas of the functional paradigm - in short, they are functions that take functions as arguments and/or return functions as results.

The need of referring to an existing `Callable` object comes up often when writing functional C++ code, but the Standard Library unfortunately doesn't provide a flexible facility that allows to do so. Let's consider the existing utilities:

- **Pointers to functions** are only useful when the entity they refer to is stateless (*i.e. a non-member function or a capture-less lambda*), but they are cumbersome to use otherwise. Fully supporting the `Callable` concept requires also explicitly dealing with **pointers to member functions** and **pointers to data members**.
- `std::function` seamlessly works with `Callable` objects, but it's a "*general-purpose polymorphic function wrapper*" that may introduce unnecessary overhead and that **owns** the `Callable` it stores. `std::function` is a great choice when an owning type-erased wrapper is required, but it's often abused when its ownership semantics and its flexibility are not required.
 - Note that when `std::function` is constructed/assigned with a `std::reference_wrapper` to a `Callable`, it has reference semantics.
 - Another limitation of `std::function` is the fact that the stored `Callable` must be `CopyConstructible`.
- **Templates** can be used to avoid unnecessary costs and to uniformly handle any `Callable` object, but they are hard to constrain to a particular signature and force code to be defined in headers.

This paper proposes the introduction of a new `function_ref` class template, which is akin to `std::string_view`. This paper describes `function_ref` as a **non-owning lightweight wrapper** over any `Callable` object.

Motivating example

Here's one example use case that benefits from *higher-order functions*: a `retry(n, f)` function that attempts to synchronously call `f` up to `n` times until success. This example might model the real-world scenario of repeatedly querying a flaky web service.

```

struct payload { /* ... */ };

// Repeatedly invokes `action` up to `times` repetitions.
// Immediately returns if `action` returns a valid `payload`.
// Returns `std::nullopt` otherwise.
std::optional<payload> retry(std::size_t times, /* ????? */ action);

```

The passed-in action should be a Callable which takes no arguments and returns `std::optional<payload>`. Let's see how `retry` can be implemented with various techniques:

- Using *pointers to functions*:

```

std::optional<payload> retry(std::size_t times,
                           std::optional<payload>(*action)())
{
    /* ... */
}

```

([on godbolt.org](https://godbolt.org/g/UQbZYp)) (<https://godbolt.org/g/UQbZYp>)

- **Advantages:**

- Easy to implement: no need to use a template or any explicit constraint (e.g. `std::enable_if_t<...>`). The type of the pointer specifies exactly which functions can be passed, no extra constraints are required.
- Minimal overhead: no allocations, no exceptions, and `action` is as big as a pointer.
 - Modern compilers are able to completely inline the call to `action`, producing optimal assembly.

- **Drawbacks:**

- This technique doesn't support stateful Callable objects.

- Using a template:

```

template <typename F>
auto retry(std::size_t times, F&& action)
    -> std::enable_if_t<std::is_invocable_r_v<std::optional<payload>, F&&>,
                    std::optional<payload>>

{
    /* ... */
}

```

([on godbolt.org](https://godbolt.org/g/AGikkz)) (<https://godbolt.org/g/AGikkz>)

- **Advantages:**

- Supports arbitrary `Callable` objects, such as stateful closures.
- Zero-overhead: no allocations, no exceptions, no indirections.

- **Drawbacks:**

- Harder to implement and less readable: users must use `std::enable_if_t` and `std::invocable_r_v` to ensure that `action`'s signature is properly constrained.
- `retry` must be defined in a header file. This might be undesirable when trying to minimize compilation times.

- Using `std::function`:

```
std::optional<payload> retry(std::size_t times,
                             std::function<std::optional<payload>()> action)
{
    /* ... */
}
```

([on godbolt.org](https://godbolt.org/g/t9FH9b)) (<https://godbolt.org/g/t9FH9b>)

- **Advantages:**

- Supports arbitrary `Callable` objects, such as stateful closures.
- Easy to implement: no need to use a `template` or any explicit constraint. The type fully constrains what can be passed.

- **Drawbacks:**

- Unclear ownership semantics: `action` might either own the the stored `Callable`, or just refer to an existing `Callable` if initialized with a `std::reference_wrapper`.
- Can potentially have significant overhead:
 - Even though the implementation makes use of SBO (*small buffer optimization*), `std::function` might allocate if the stored object is large enough. This requires one extra branch on construction/assignment, one potential dynamic allocation, and makes `action` as big as the size of the internal buffer.
 - If the implementation doesn't make use of SBO, `std::function` will always allocate on construction/assignment.

- Modern compilers are not able to inline `std::function`, often resulting in very poor assembly compared to the previously mentioned techniques.
 - Mandatory use of exceptions: `std::function` might throw if an allocation fails, and throws `std::bad_function_call` if it's invoked while unset.
- Using the proposed `function_ref`:

```
std::optional<payload> retry(std::size_t times,
                           function_ref<std::optional<payload>()> action)
{
    /* ... */
}
```

(*on godbolt.org*) (<https://godbolt.org/g/DvWKVH>)

- **Advantages:**

- Supports arbitrary `Callable` objects, such as stateful closures.
 - Easy to implement: no need to use a template or any constraint. The type fully constrains what can be passed.
 - Clear ownership semantics: `action` is a **non-owning** reference to an existing `Callable`.
 - Small overhead: no allocations, no exceptions, and `action` is as big as two pointers.
 - Modern compilers are able to completely inline the call to `action`, producing optimal assembly.

Impact on the Standard

This proposal is a pure library extension. It does not require changes to any existing part of the Standard.

Alternatives

The only existing viable alternative to `function_ref` currently is `std::function + std::reference_wrapper`. The Standard guarantees that when a `std::reference_wrapper` is used to construct/assign to a `std::function` no allocations will occur and no exceptions will be thrown.

Using `std::function` for non-owning references is suboptimal for various reasons.

1. The ownership semantics of a `std::function` are unclear - they change depending on whether or not the `std::function` was constructed/assigned with a `std::reference_wrapper`.

```
void foo(std::function<void()> f);  
// `f` could be referring to an existing Callable, or could own one.  
  
void bar(function_ref<void()> f);  
// `f` unambiguously is a non-owning reference to an existing Callable.
```

2. This technique doesn't work with temporaries. This is a huge drawback as it prevents stateful temporary lambdas from being passed as callbacks.

```
void foo(std::function<void()> f);  
  
int main()  
{  
    int x = 0;  
    foo(std::ref([&x]{ ++x; })); // does not compile  
}
```

(on [godbolt.org](https://godbolt.org/g/DPQ7ku)) (<https://godbolt.org/g/DPQ7ku>)

The code above doesn't compile, as `std::ref` only accepts non-const lvalue references (*additionally, `std::cref` is explicitly deleted for rvalue references*). Avoiding the use of `std::ref` breaks the guarantee that `f` won't allocate or throw an exception on construction.

3. `std::function` is harder for compilers to optimize compared to the proposed `function_ref`. This is true due to various reasons:

- `std::function` can allocate and/or throw exceptions on construction and/or assignment.
- `std::function` might use SBO, which could require an additional branch during construction/assignment, make inlining more difficult, and unnecessarily increase memory usage.

Rough benchmarks comparing the generated assembly of a `std::function` parameter and a `function_ref` parameter against a `template` parameter show that:

- `std::function`, on average, generates approximately 5x more assembly than a template parameter.
- `function_ref`, on average, generates approximately 1.5x more assembly than a template parameter.

A description of the benchmarking techniques used and the full results can be found on my article "[passing functions to functions](#)"¹.

Synopsis

```
namespace std
{
    template <typename>
    class function_ref; /* undefined */

    template <typename R, typename... Args>
    class function_ref<R(Args...)>
    {
    public:
        constexpr function_ref() noexcept;
        constexpr function_ref(std::nullptr_t) noexcept;
        constexpr function_ref(const function_ref&) noexcept;

        template <typename F>
        constexpr function_ref(F&&) noexcept;

        constexpr function_ref& operator=(const function_ref&) noexcept;
        constexpr function_ref& operator=(std::nullptr_t) noexcept;

        template <typename F>
        constexpr function_ref& operator=(F&&) noexcept;

        constexpr void swap(function_ref&) noexcept;

        constexpr explicit operator bool() const noexcept;

        R operator()(Args...) const;
    };

    template <typename R, typename... Args>
    constexpr void swap(function_ref<R(Args...)>&, function_ref<R(Args...)>&) noexcept;

    template <typename R, typename... Args>
    constexpr bool operator==(const function_ref<R(Args...)>&, std::nullptr_t) noexcept;

    template <typename R, typename... Args>
    constexpr bool operator==(std::nullptr_t, const function_ref<R(Args...)>&) noexcept;

    template <typename R, typename... Args>
    constexpr bool operator!=(const function_ref<R(Args...)>&, std::nullptr_t) noexcept;

    template <typename R, typename... Args>
    constexpr bool operator!=(std::nullptr_t, const function_ref<R(Args...)>&) noexcept;
```

```

template <typename R, typename... Args>
function_ref(R (*)(Args...)) -> function_ref<R(Args...)>;

template <typename F>
function_ref(F) -> function_ref</* deduced if possible */>;
}

```

Specification

```

template <typename R, typename... Args>
constexpr function_ref<R(Args...)>::function_ref() noexcept;

```

- *Effects*: constructs a function_ref referring to nothing.
- *Postconditions*: !*this.

```

template <typename R, typename... Args>
constexpr function_ref<R(Args...)>::function_ref(std::nullptr_t) noexcept;

```

- *Effects*: constructs a function_ref referring to nothing.
- *Postconditions*: !*this.

```

template <typename R, typename... Args>
constexpr function_ref<R(Args...)>::function_ref(const function_ref& rhs) noexcept;

```

- *Effects*: constructs a function_ref referring to the same callable rhs refers to (or to nothing if bool(rhs) == false).
- *Postconditions*: !*this if !rhs; otherwise, *this refers to the same Callable rhs refers to.


```

template <typename R, typename... Args>
template <typename F>
constexpr function_ref<R(Args...)>::function_ref(F&& f) noexcept;

```

- *Remarks:* This function shall not participate in overload resolution unless
`!std::is_same_v<std::decay_t<F>, function_ref> && std::is_invocable_v<F&&, R, Args...>`.
- *Effects:* constructs a `function_ref` referring to `f`.
- *Postconditions:* `!*this` if any of the following hold:
 - `f` is a null function pointer value.
 - `f` is a null member pointer value.

Otherwise, `*this` refers to `f`.

```

template <typename R, typename... Args>
constexpr function_ref& function_ref<R(Args...)>::operator=(const function_ref& rhs) noexcept;

```

- *Effects:* `*this` refers to the same callable `rhs` refers to (or to nothing if `bool(rhs) == false`).
- *Postconditions:* `bool(*this) == bool(rhs)`.
- *Returns:* `*this`.

```

template <typename R, typename... Args>
constexpr function_ref& function_ref<R(Args...)>::operator=(std::nullptr_t) noexcept;

```

- *Effects:* `*this` refers to nothing.
- *Postconditions:* `!*this`.
- *Returns:* `*this`.

```
template <typename R, typename... Args>
template <typename F>
constexpr function_ref& function_ref<R(Args...)>::operator=(F&&) noexcept;
```

- *Requires:* `std::is_invocable_v<F&&, R, Args...>`.
- *Effects:* `*this` refers to `f`.
- *Postconditions:* `bool(*this) == true`.
- *Returns:* `*this`.

```
template <typename R, typename... Args>
constexpr void function_ref<R(Args...)>::swap(function_ref& rhs) noexcept;
```

- *Effects:* exchanges the values of `*this` and `rhs`.

```
template <typename R, typename... Args>
constexpr explicit function_ref<R(Args...)>::operator bool() const noexcept;
```

- *Returns:* `true` if `*this` is referring to something, `false` otherwise.

```
template <typename R, typename... Args>
R function_ref<R(Args...)>::operator()(Args... xs) const;
```

- *Requires:* `bool(*this) == true`.
- *Effects:* equivalent to `return INVOKE<R>(f, std::forward<Args>(xs)...);`, where `f` is the function object referred to by `*this`.

```
template <typename R, typename... Args>
constexpr void swap(function_ref<R(Args...)>& lhs, function_ref<R(Args...)>& rhs) noexcept;
```

- *Effects*: equivalent to `lhs.swap(rhs)`.

```
template <typename R, typename... Args>
constexpr bool operator==(const function_ref<R(Args...)>& fr, std::nullptr_t) noexcept;
```

- *Returns*: false if `fr` is referring to something, true otherwise.

```
template <typename R, typename... Args>
constexpr bool operator==(std::nullptr_t, const function_ref<R(Args...)>& fr) noexcept;
```

- *Returns*: `fr == nullptr`.

```
template <typename R, typename... Args>
constexpr bool operator!=(const function_ref<R(Args...)>& fr, std::nullptr_t) noexcept;
```

- *Returns*: `!(fr == nullptr)`.

```
template <typename R, typename... Args>
constexpr bool operator!=(std::nullptr_t fr, const function_ref<R(Args...)>&) noexcept;
```

- *Returns*: `!(fr == nullptr)`.

Existing practice

Many facilities similar to `function_ref` exist and are widely used in large codebases. Here are some examples:

- The `llvm::function_ref`² class template is used throughout LLVM. A quick GitHub search on the LLVM organization reports hundreds of usages both in `llvm` and `clang`³.
- Facebook's Folly libraries⁴ provide a `folly::FunctionRef`⁵ class template. A GitHub search shows that it's used in projects `proxygen` and `fbthrift`⁶.
- GNU's popular debugger, `gdb`⁷, uses `gdb::function_view`⁸ throughout its code base. The documentation in the linked header file⁹ is particularly well-written and greatly motivates the need for this facility.

Additionally, combining results from GitHub searches (*excluding "llvm" and "folly"*) for "`function_ref`"¹⁰, "`function_view`"¹¹, "`FunctionRef`"¹², and "`FunctionView`"¹³ roughly shows more than 2800 occurrences.

Possible issues

Accepting temporaries in `function_ref`'s constructor is extremely useful in the most common use case: using it as a function parameter:

```
void foo(function_ref<void()>);

int main()
{
    foo([]{ });
}
```

(*on wandbox.org*) (<https://wandbox.org/permlink/BPtbPeQtErPGj4X7>)

The usage shown above is completely safe: the temporary closure generated by the lambda expression is guarantee to live for the entirety of the call to `foo`. Unfortunately, this also means that the following code snippet will result in *undefined behavior*:

```
int main()
{
    function_ref<void()> f([]{ });
    // ...
    f(); // undefined behavior
}
```

(*on wandbox.org*) (<https://wandbox.org/permlink/cQPEX2sKjCQjg1ki>)

The above closure is a temporary whose lifetime ends after the `function_ref` constructor call. The `function_ref` will store an address to a "dead" closure - invoking it will produce undefined behavior ¹⁴. As an example, `AddressSanitizer` detects an invalid memory access in this gist ¹⁵. Note that this problem is not unique to `function_ref`: the recently standardized `std::string_view` ¹⁶ has the same problem ¹⁷.

I strongly believe that accepting temporaries is a "necessary evil" for both `function_ref` and `std::string_view`, as it enables countless valid use cases. The problem of dangling references has been always present in the language - a more general solution like Herb Sutter and Neil Macintosh's lifetime tracking ¹⁸ would prevent mistakes without limiting the usefulness of view/reference classes.

Open questions

Below are some unanswered questions for which I kindly ask guidance from members of the committee and readers of this paper.

- `function_ref::operator()` is not currently marked as `constexpr` due to implementation issues. I could not figure a way to implement a `constexpr`-friendly `operator()`. Is there any possibility it could be marked as `constexpr` to increase the usefulness of `function_ref`?
- `function_ref::operator()` is qualified as `const` and always invokes the referenced `Callable` with `std::invoke(f, std::forward<Args>(xs)...)` . Should `f` be perfectly-forwarded? Should ref-qualified versions of `function_ref::operator()` that apply the ref qualifier to `f` be provided to allow users to invoke the referenced `Callable`'s potentially ref-qualified `operator()` overloads?
- As currently proposed, `function_ref` can be in an *empty state*. This choice was made to maintain consistency with existing Standard Library classes such as `std::function` and `std::string_view`. A possible alternative is designing `function_ref` without an empty state and encourage the use of `std::optional<std::function_ref<...>>` to model a possibly null reference to a `Callable`. Should `function_ref` have an empty state?
- As currently proposed, `function_ref::operator()` is unconditionally qualified as `const`, even though the referred `Callable` object's `operator()` might have different qualifiers and/or mutate its own state upon invocation. This choice was made to maintain consistency with `std::function`. There have been proposals in the past that would allow `std::function` signatures to be qualified (see P0045R1 ¹⁹ and N4159 ²⁰) in order to statically check whether or not an adopted target object is compatible with the specified qualifiers. Should `function_ref` embrace the ideas discussed in P0045R1, achieving more type-safety at the cost of breaking consistency with `std::function`?

Bikeshedding

The name `function_ref` is subject to bikeshedding. Here are some other potential names:

- `function_view`
- `callable_ref`
- `callable_view`
- `invocable_ref`
- `invocable_view`
- `fn_view`
- `fn_ref`

Acknowledgments

Thanks to **Eric Niebler**, **Tim van Deurzen**, and **Alisdair Meredith** for providing very valuable feedback on earlier drafts of this proposal.

References

1. https://vittorioromeo.info/index/blog/passing_functions_to_functions.html#benchmark---generated-assembly (https://vittorioromeo.info/index/blog/passing_functions_to_functions.html#benchmark---generated-assembly)↵
2. http://llvm.org/doxygen/classllvm_1_1function_ref_3_01Ret_07Params_8_8_8_08_4.html (http://llvm.org/doxygen/classllvm_1_1function_ref_3_01Ret_07Params_8_8_8_08_4.html)↵
3. https://github.com/search?q=org%3Allvm-mirror+function_ref&type=Code (https://github.com/search?q=org%3Allvm-mirror+function_ref&type=Code)↵
4. <https://github.com/facebook/folly> (<https://github.com/facebook/folly>)↵
5. <https://github.com/facebook/folly/blob/master/folly/Function.h#L743-L824> (<https://github.com/facebook/folly/blob/master/folly/Function.h#L743-L824>)↵

6. <https://github.com/search?q=org%3Afacebook+FunctionRef&type=Code> (<https://github.com/search?q=org%3Afacebook+FunctionRef&type=Code>)↵
7. <https://www.gnu.org/software/gdb/> (<https://www.gnu.org/software/gdb/>)↵
8. <https://sourceware.org/git/gitweb.cgi?p=binutils-gdb.git;a=blob;f=gdb/common/function-view.h> (<https://sourceware.org/git/gitweb.cgi?p=binutils-gdb.git;a=blob;f=gdb/common/function-view.h>)↵
9. <https://sourceware.org/git/gitweb.cgi?p=binutils-gdb.git;a=blob;f=gdb/common/function-view.h> (<https://sourceware.org/git/gitweb.cgi?p=binutils-gdb.git;a=blob;f=gdb/common/function-view.h>)↵
10. https://github.com/search?utf8=%E2%9C%93&q=function_ref+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code (https://github.com/search?utf8=%E2%9C%93&q=function_ref+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code)↵
11. https://github.com/search?utf8=%E2%9C%93&q=function_view+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code (https://github.com/search?utf8=%E2%9C%93&q=function_view+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code)↵
12. <https://github.com/search?utf8=%E2%9C%93&q=functionref+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code> (<https://github.com/search?utf8=%E2%9C%93&q=functionref+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code>)↵
13. <https://github.com/search?utf8=%E2%9C%93&q=functionview+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code> (<https://github.com/search?utf8=%E2%9C%93&q=functionview+AND+NOT+llvm+AND+NOT+folly+language%3AC%2B%2B&type=Code>)↵
14. <http://foonathan.net/blog/2017/01/20/function-ref-implementation.html> (<http://foonathan.net/blog/2017/01/20/function-ref-implementation.html>)↵
15. <https://gist.github.com/SuperV1234/a41eb1c825bfbb43f595b13bd4ea99c3> (<https://gist.github.com/SuperV1234/a41eb1c825bfbb43f595b13bd4ea99c3>)↵
16. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3762.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3762.html>)↵
17. http://foonathan.net/blog/2017/03/22/string_view-temporary.html (http://foonathan.net/blog/2017/03/22/string_view-temporary.html)↵
18. <https://github.com/isocpp/CppCoreGuidelines/blob/master/docs/Lifetimes%20I%20and%20II%20-%20v0.9.1.pdf> (<https://github.com/isocpp/CppCoreGuidelines/blob/master/docs/Lifetimes%20I%20and%20II%20-%20v0.9.1.pdf>)↵

19. <http://wg21.link/p0045r1> (<http://wg21.link/p0045r1>)↵

20. <http://wg21.link/N4159> (<http://wg21.link/N4159>)↵